第 23 章

JSON

本章内容

- 理解 JSON 语法
- 解析 JSON
- JSON 序列化

正如上一章所说，XML 曾经一度成为互联网上传输数据的事实标准。第一代 Web 服务很大程度上是以 XML 为基础的，以服务器间通信为主要特征。可是，XML 也并非没有批评者。有的人认为 XML 过于冗余和啰唆。为解决这些问题，也出现了几种方案。不过 Web 已经朝着它的新方向进发了。

2006 年，Douglas Crockford 在国际互联网工程任务组（IETF，The Internet Engineering Task Force）制定了 JavaScript 对象简谱（JSON，JavaScript Object Notation）标准，即 RFC 4627。但实际上，JSON 早在 2001 年就开始使用了。JSON 是 JavaScript 的严格子集，利用 JavaScript 中的几种模式来表示结构化数据。Crockford 将 JSON 作为替代 XML 的一个方案提出，因为 JSON 可以直接传给 `eval()` 而不需要创建 DOM。

理解 JSON 最关键的一点是要把它当成一种数据格式，而不是编程语言。JSON 不属于 JavaScript，它们只是拥有相同的语法而已。JSON 也不是只能在 JavaScript 中使用，它是一种通用数据格式。很多语言都有解析和序列化 JSON 的内置能力。

23.1 语法

JSON 语法支持表示 3 种类型的值。

- **简单值**：字符串、数值、布尔值和 `null` 可以在 JSON 中出现，就像在 JavaScript 中一样。特殊值 `undefined` 不可以。
- **对象**：第一种复杂数据类型，对象表示有序键/值对。每个值可以是简单值，也可以是复杂类型。
- **数组**：第二种复杂数据类型，数组表示可以通过数值索引访问的值的有序列表。数组的值可以是任意类型，包括简单值、对象，甚至其他数组。

JSON 没有变量、函数或对象实例的概念。JSON 的所有记号都只为表示结构化数据，虽然它借用了 JavaScript 的语法，但是千万不要把它跟 JavaScript 语言混淆。

23.1.1 简单值

最简单的 JSON 可以是一个数值。例如，下面这个数值是有效的 JSON：

这个 JSON 表示数值 5。类似地，下面这个字符串也是有效的 JSON：

```
"Hello world!"
```

JavaScript 字符串与 JSON 字符串的主要区别是，JSON 字符串必须使用双引号（单引号会导致语法错误）。

布尔值和 `null` 本身也是有效的 JSON 值。不过，实践中更多使用 JSON 表示比较复杂的数据结构，其中会包含简单值。

23.1.2　对象

对象使用与 JavaScript 对象字面量略为不同的方式表示。以下是 JavaScript 中的对象字面量：

```
let person = {
  name: "Nicholas",
  age: 29
};
```

虽然这对 JavaScript 开发者来说是标准的对象字面量，但 JSON 中的对象必须使用双引号把属性名包围起来。下面的代码与前面的代码是一样的：

```
let object = {
  "name": "Nicholas",
  "age" : 29
};
```

而用 JSON 表示相同的对象的语法是：

```
{
  "name": "Nicholas",
  "age": 29
}
```

与 JavaScript 对象字面量相比，JSON 主要有两处不同。首先，没有变量声明（JSON 中没有变量）。其次，最后没有分号（不需要，因为不是 JavaScript 语句）。同样，用引号将属性名包围起来才是有效的 JSON。属性的值可以是简单值或复杂数据类型值，后者可以在对象中再嵌入对象，比如：

```
{
  "name": "Nicholas",
  "age": 29,
  "school": {
    "name": "Merrimack College",
    "location": "North Andover, MA"
  }
}
```

这个例子在顶级对象中又嵌入了学校相关的信息。即使整个 JSON 对象中有两个属性都叫 `"name"`，但它们属于两个不同的对象，因此是允许的。同一个对象中不允许出现两个相同的属性。

与 JavaScript 不同，JSON 中的对象属性名必须始终带双引号。手动编写 JSON 时漏掉这些双引号或使用单引号是常见错误。

23.1.3　数组

JSON 的第二种复杂数据类型是数组。数组在 JSON 中使用 JavaScript 的数组字面量形式表示。例如，以下是一个 JavaScript 数组：

```
let values = [25, "hi", true];
```

在 JSON 中可以使用类似语法表示相同的数组：

```
[25, "hi", true]
```

同样，这里没有变量，也没有分号。数组和对象可以组合使用，以表示更加复杂的数据结构，比如：

```
[
  {
    "title": "Professional JavaScript",
    "authors": [
      "Nicholas C. Zakas",
      "Matt Frisbie"
    ],
    "edition": 4,
    "year": 2017
  },
  {
    "title": "Professional JavaScript",
    "authors": [
      "Nicholas C. Zakas"
    ],
    "edition": 3,
    "year": 2011
  },
  {
    "title": "Professional JavaScript",
    "authors": [
      "Nicholas C. Zakas"
    ],
    "edition": 2,
    "year": 2009
  },
  {
    "title": "Professional Ajax",
    "authors": [
      "Nicholas C. Zakas",
      "Jeremy McPeak",
      "Joe Fawcett"
    ],
    "edition": 2,
    "year": 2008
  },
  {
    "title": "Professional Ajax",
    "authors": [
      "Nicholas C. Zakas",
      "Jeremy McPeak",
      "Joe Fawcett"
    ],
    "edition": 1,
    "year": 2007
  },
  {
    "title": "Professional JavaScript",
    "authors": [
      "Nicholas C. Zakas"
    ],
```

```
    "edition": 1,
    "year": 2006
  }
]
```

前面这个数组包含了很多表示书的对象。每个对象都包含一些键，其中一个是"authors"，对应的值也是一个数组。对象和数组通常会作为 JSON 数组的顶级结构（尽管不是必需的），以便创建大型复杂数据结构。

23.2　解析与序列化

JSON 的迅速流行并不仅仅因为其语法与 JavaScript 类似，很大程度上还因为 JSON 可以直接被解析成可用的 JavaScript 对象。与解析为 DOM 文档的 XML 相比，这个优势非常明显。为此，JavaScript 开发者可以非常方便地使用 JSON 数据。比如，前面例子中的 JSON 包含很多图书，通过如下代码就可以获取第三本书的书名：

```
books[2].title
```

当然，以上代码假设把前面的数据结构保存在了变量 books 中。相比之下，遍历 DOM 结构就显得麻烦多了：

```
doc.getElementsByTagName("book")[2].getAttribute("title");
```

看看这些方法调用，就不难想象为什么 JSON 大受 JavaScript 开发者欢迎了。JSON 出现之后就迅速成为了 Web 服务的事实序列化标准。

23.2.1　JSON 对象

早期的 JSON 解析器基本上就相当于 JavaScript 的 eval()函数。因为 JSON 是 JavaScript 语法的子集，所以 eval()可以解析、解释，并将其作为 JavaScript 对象和数组返回。ECMAScript 5 增加了 JSON 全局对象，正式引入解析 JSON 的能力。这个对象在所有主流浏览器中都得到了支持。旧版本的浏览器可以使用垫片脚本（参见 GitHub 上 douglascrockford/JSON-js 中的 JSON in JavaScript）。考虑到直接执行代码的风险，最好不要在旧版本浏览器中只使用 eval()求值 JSON。这个 JSON 垫片脚本最好只在浏览器原生不支持 JSON 解析时使用。

JSON 对象有两个方法：stringify()和 parse()。在简单的情况下，这两个方法分别可以将 JavaScript 序列化为 JSON 字符串，以及将 JSON 解析为原生 JavaScript 值。例如：

```
let book = {
  title: "Professional JavaScript",
  authors: [
    "Nicholas C. Zakas",
    "Matt Frisbie"
  ],
  edition: 4,
  year: 2017
};
let jsonText = JSON.stringify(book);
```

这个例子使用 JSON.stringify()把一个 JavaScript 对象序列化为一个 JSON 字符串，保存在变量 jsonText 中。默认情况下，JSON.stringify()会输出不包含空格或缩进的 JSON 字符串，因此

jsonText 的值是这样的：

```
{"title":"Professional JavaScript","authors":["Nicholas C. Zakas","Matt Frisbie"],
"edition":4,"year":2017}
```

在序列化 JavaScript 对象时，所有函数和原型成员都会有意地在结果中省略。此外，值为 undefined 的任何属性也会被跳过。最终得到的就是所有实例属性均为有效 JSON 数据类型的表示。

JSON 字符串可以直接传给 JSON.parse()，然后得到相应的 JavaScript 值。比如，可以使用以下代码创建与 book 对象类似的新对象：

```
let bookCopy = JSON.parse(jsonText);
```

注意，book 和 bookCopy 是两个完全不同的对象，没有任何关系。但是它们拥有相同的属性和值。

如果给 JSON.parse()传入的 JSON 字符串无效，则会导致抛出错误。

23.2.2 序列化选项

实际上，JSON.stringify()方法除了要序列化的对象，还可以接收两个参数。这两个参数可以用于指定其他序列化 JavaScript 对象的方式。第一个参数是过滤器，可以是数组或函数；第二个参数是用于缩进结果 JSON 字符串的选项。单独或组合使用这些参数可以更好地控制 JSON 序列化。

1. 过滤结果

如果第二个参数是一个数组，那么 JSON.stringify()返回的结果只会包含该数组中列出的对象属性。比如下面的例子：

```
let book = {
  title: "Professional JavaScript",
  authors: [
    "Nicholas C. Zakas",
    "Matt Frisbie"
  ],
  edition: 4,
  year: 2017
};
let jsonText = JSON.stringify(book, ["title", "edition"]);
```

在这个例子中，JSON.stringify()方法的第二个参数是一个包含两个字符串的数组："title"和"edition"。它们对应着要序列化的对象中的属性，因此结果 JSON 字符串中只会包含这两个属性：

```
{"title":"Professional JavaScript","edition":4}
```

如果第二个参数是一个函数，则行为又有不同。提供的函数接收两个参数：属性名（key）和属性值（value）。可以根据这个 key 决定要对相应属性执行什么操作。这个 key 始终是字符串，只是在值不属于某个键/值对时会是空字符串。

为了改变对象的序列化，返回的值就是相应 key 应该包含的结果。注意，返回 undefined 会导致属性被忽略。下面看一个例子：

```
let book = {
  title: "Professional JavaScript",
  authors: [
    "Nicholas C. Zakas",
    "Matt Frisbie"
  ],
  edition: 4,
```

```
  year: 2017
};
let jsonText = JSON.stringify(book, (key, value) => {
  switch(key) {
    case "authors":
      return value.join(",")
    case "year":
      return 5000;
    case "edition":
      return undefined;
    default:
      return value;
  }
});
```

这个函数基于键进行了过滤。如果键是"authors"，则将数组值转换为字符串；如果键是"year"，则将值设置为 5000；如果键是"edition"，则返回 undefined 忽略该属性。最后一定要提供默认返回值，以便返回其他属性传入的值。第一次调用这个函数实际上会传入空字符串 key，值是 book 对象。最终得到的 JSON 字符串是这样的：

```
{"title":"Professional JavaScript","authors":"Nicholas C. Zakas,Matt
Frisbie","year":5000}
```

注意，函数过滤器会应用到要序列化的对象所包含的所有对象，因此如果数组中包含多个具有这些属性的对象，则序列化之后每个对象都只会剩下上面这些属性。

Firefox 3.5~3.6 在 JSON.stringify()的第二个参数是函数时有一个 bug：此时函数只能作为过滤器，返回 undefined 会导致跳过属性，返回其他值则会包含属性。Firefox 4 修复了这个 bug。

2. 字符串缩进

JSON.stringify()方法的第三个参数控制缩进和空格。在这个参数是数值时，表示每一级缩进的空格数。例如，每级缩进 4 个空格，可以这样：

```
let book = {
  title: "Professional JavaScript",
  authors: [
    "Nicholas C. Zakas",
    "Matt Frisbie"
  ],
  edition: 4,
  year: 2017
};
let jsonText = JSON.stringify(book, null, 4);
```

这样得到的 jsonText 格式如下：

```
{
    "title": "Professional JavaScript",
    "authors": [
        "Nicholas C. Zakas",
        "Matt Frisbie"
    ],
    "edition": 4,
    "year": 2017
}
```

注意，除了缩进，JSON.stringify()方法还为方便阅读插入了换行符。这个行为对于所有有效的

缩进参数都会发生。（只缩进不换行也没什么用。）最大缩进值为 10，大于 10 的值会自动设置为 10。

如果缩进参数是一个字符串而非数值，那么 JSON 字符串中就会使用这个字符串而不是空格来缩进。使用字符串，也可以将缩进字符设置为 Tab 或任意字符，如两个连字符：

```
let jsonText = JSON.stringify(book, null, "--" );
```

这样，jsonText 的值会变成如下格式：

```
{
--"title": "Professional JavaScript",
--"authors": [
----"Nicholas C. Zakas",
----"Matt Frisbie"
--],
--"edition": 4,
--"year": 2017
}
```

使用字符串时同样有 10 个字符的长度限制。如果字符串长度超过 10，则会在第 10 个字符处截断。

3. toJSON()方法

有时候，对象需要在 JSON.stringify()之上自定义 JSON 序列化。此时，可以在要序列化的对象中添加 toJSON()方法，序列化时会基于这个方法返回适当的 JSON 表示。事实上，原生 Date 对象就有一个 toJSON()方法，能够自动将 JavaScript 的 Date 对象转换为 ISO 8601 日期字符串（本质上与在 Date 对象上调用 toISOString()方法一样）。

下面的对象为自定义序列化而添加了一个 toJSON()方法：

```
let book = {
  title: "Professional JavaScript",
  authors: [
    "Nicholas C. Zakas",
    "Matt Frisbie"
  ],
  edition: 4,
  year: 2017,
  toJSON: function() {
    return this.title;
  }
};
let jsonText = JSON.stringify(book);
```

这里 book 对象中定义的 toJSON()方法简单地返回了图书的书名（this.title）。与 Date 对象类似，这个对象会被序列化为简单字符串而非对象。toJSON()方法可以返回任意序列化值，都可以起到相应的作用。如果对象被嵌入在另一个对象中，返回 undefined 会导致值变成 null；或者如果是顶级对象，则本身就是 undefined。注意，箭头函数不能用来定义 toJSON()方法。主要原因是箭头函数的词法作用域是全局作用域，在这种情况下不合适。

toJSON()方法可以与过滤函数一起使用，因此理解不同序列化流程的顺序非常重要。在把对象传给 JSON.stringify()时会执行如下步骤。

(1) 如果可以获取实际的值，则调用 toJSON()方法获取实际的值，否则使用默认的序列化。

(2) 如果提供了第二个参数，则应用过滤。传入过滤函数的值就是第(1)步返回的值。

(3) 第(2)步返回的每个值都会相应地进行序列化。

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权

(4) 如果提供了第三个参数，则相应地进行缩进。

理解这个顺序有助于决定是创建 toJSON()方法，还是使用过滤函数，抑或是两者都用。

23.2.3　解析选项

JSON.parse()方法也可以接收一个额外的参数，这个函数会针对每个键/值对都调用一次。为区别于传给 JSON.stringify()的起过滤作用的**替代函数**（replacer），这个函数被称为**还原函数**（reviver）。实际上它们的格式完全一样，即还原函数也接收两个参数，属性名（key）和属性值（value），另外也需要返回值。

如果还原函数返回 undefined，则结果中就会删除相应的键。如果返回了其他任何值，则该值就会成为相应键的值插入到结果中。还原函数经常被用于把日期字符串转换为 Date 对象。例如：

```
let book = {
  title: "Professional JavaScript",
  authors: [
    "Nicholas C. Zakas",
    "Matt Frisbie"
  ],
  edition: 4,
  year: 2017,
  releaseDate: new Date(2017, 11, 1)
};
let jsonText = JSON.stringify(book);
let bookCopy = JSON.parse(jsonText,
    (key, value) => key == "releaseDate" ? new Date(value) : value);
alert(bookCopy.releaseDate.getFullYear());
```

以上代码在 book 对象中增加了 releaseDate 属性，是一个 Date 对象。这个对象在被序列化为 JSON 字符串后，又被重新解析为一个对象 bookCopy。这里的还原函数会查找"releaseDate"键，如果找到就会根据它的日期字符串创建新的 Date 对象。得到的 bookCopy.releaseDate 属性又变回了 Date 对象，因此可以调用其 getFullYear()方法。

23.3　小结

JSON 是一种轻量级数据格式，可以方便地表示复杂数据结构。这个格式使用 JavaScript 语法的一个子集表示对象、数组、字符串、数值、布尔值和 null。虽然 XML 也能胜任同样的角色，但 JSON 更简洁，JavaScript 支持也更好。更重要的是，所有浏览器都已经原生支持全局 JSON 对象。

ECMAScript 5 定义了原生 JSON 对象，用于将 JavaScript 对象序列化为 JSON 字符串，以及将 JSON 数组解析为 JavaScript 对象。JSON.stringify()和 JSON.parse()方法分别用于实现这两种操作。这两个方法都有一些选项可以用来改变默认的行为，以实现过滤或修改流程。

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权